

LEMMA: Learned Guidance for Evidence-Carrying Long-Horizon Symbolic Rewriting

Atul Saxena

atul@blackdrome.tech

BlackdromeAI Labs

Pushp Kharat

pushp@pkboost.co.in

Pkboost AI Labs

September 2026

With contributions from Divyanshi Sharma, BlackdromeAI Labs divyanshi@blackdrome.tech

OPEN RESOURCES [Code repository](#) | [Benchmark dataset](#) | [Released v3 model](#) | [Paper DOI](#)

Abstract

Symbolic mathematics systems can often verify an individual algebraic rewrite cheaply. The harder problem is selecting a long sequence of valid rewrites that transforms a starting expression into a specified goal. This becomes difficult when an intermediate expression has several verifier-admitted moves, because a locally plausible choice can lead away from the target. LEMMA is a neuro-symbolic system built for this setting. It combines a Rust symbolic rule engine with a guardrail that filters candidate actions, an evidence-typed verifier that checks every accepted transition, and a neural policy that ranks the remaining choices. The design separates two questions that are often conflated: which transformations are valid, and which valid transformation is most useful for reaching the goal. Each returned derivation also carries an explicit symbolic, numerical, or replay-only evidence status.

We evaluate six search and decoding procedures on a frozen 120-problem long-horizon benchmark, with reference derivations of depths 64, 96, and 128. Eighty-nine problems contain genuine decision points rather than forced rewrite corridors. All methods receive the same guardrail-filtered, verifier-admitted successors, allowing the experiment to isolate the contribution of learned guidance. Uniform MCTS and value-only MCTS solve none of these 89 problems. At a 400 generated-successor budget, width-8 beam search also solves none, although it reaches all 89 when its budget increases to 1600. In contrast, policy-value MCTS solves 61/89, policy-only MCTS solves 87/89, and greedy decoding with the learned policy solves all 89/89 without tree search.

These results show that, on this controlled benchmark, learned local action ranking is the most budget-efficient component of the system. We additionally report a matched three-seed target-conditioning study, analyze the present value head, and release the complete experimental harness so that the reported derivations and evaluation protocol can be reproduced.

1 Introduction

Symbolic computer algebra systems can check an individual rewrite cheaply. The hard part is choosing a sequence of rewrites that reaches a particular target. For small expressions, brute force or a hand-written heuristic is usually enough. It is not enough when a derivation needs dozens of correct choices and each state has several legal moves. In the branching regime studied here, exhaustive search grows exponentially with depth even though individual steps remain cheap to check.

LEMMA couples a Rust rule engine with a guardrail that removes implausible actions, an evidence-typed verifier, and learned action guidance. We evaluate greedy decoding, beam search, and PUCT-style MCTS over the same verifier-admitted successors. Each reported derivation is replayed against the live rule engine and retains its evidence status (Section 3). Numerical checks use bounded sampling, so they provide evidence rather than a proof certificate.

The paper has four contributions.

First, we implement a verifier-constrained neuro-symbolic rewriting system: learned components rank only guardrail-filtered, verifier-accepted actions, and every returned derivation carries explicit evidence provenance.

Second, we introduce a controlled 120-problem, long-horizon evaluation at depths 64–128. Each reference path is replayed exactly and each state’s branching factor is measured directly. On the 89 problems with genuine choices, greedy learned decoding solves all 89, policy-only MCTS solves 87, and policy-value MCTS solves 61; uniform and value-only MCTS solve none. Width-8 beam search rises from 0/89 at budget 400 to 89/89 at budget 1600. This controlled component comparison is the central empirical result.

Third, a six-way decision-procedure comparison and beam-budget curve identify learned action priors as the most budget-efficient component: greedy policy execution solves all 89 branching problems, while beam requires a fourfold larger generated-successor budget to do the same.

Fourth, we measure verified branching at every state in every reference solution, separating forced corridors from genuinely decision-bearing cases.

The scope is deliberately narrow. We do not claim state-of-the-art mathematical reasoning or compare against computer algebra systems and theorem provers on their own benchmarks. The rule vocabulary covers seventeen algebraic, trigonometric, and linear-equation template families (Table 4), and evaluation uses an expert-reviewed, purpose-built controlled benchmark rather than an external general-mathematics leaderboard. Within this setting, the ablation identifies learned local action ranking as the most budget-efficient component. The current value head does not improve on direct policy execution, while beam search reaches the same endpoint only at a substantially larger budget.

2 Related Work

Symbolic computation and learned symbolic mathematics. General-purpose computer algebra systems such as SymPy [Meurer et al., 2017] apply a fixed library of simplification and solving procedures under hand-tuned heuristics rather than learned search; LEMMA’s rule engine and verifier occupy a similar role as a trusted, checkable substrate, but the question this paper asks is upstream of that substrate: given a correct set of rules, how does a system decide *which* one to apply at each of many steps. A separate line of work trains a sequence model to emit a symbolic result directly, with no verified intermediate derivation: transformers that predict an integral or ODE solution in one shot [Lample and Charton, 2020], that compute linear-algebra operations end to end [Charton, 2022], that regress a closed-form recurrence from data [d’Ascoli et al., 2022], that synthesize and execute an external program [Drori et al., 2022], or that are scored purely on final-answer match with no notion of a checkable step at all [Saxton et al., 2019]. Davies et al. [2021] instead uses a trained model’s attributions to guide a human toward a conjecture, with verification happening entirely outside the system as subsequent human proof. None of these commits to a sequence of independently verified rewrite steps toward a specified target expression, which is the setting LEMMA is built for.

Term rewriting, rewrite strategies, and equality saturation. Classical term-rewriting theory [Knuth and Bendix, 1970, Dershowitz, 1987, Baader and Nipkow, 1998, Bezem et al., eds., 2003] establishes confluence and termination for a rule *set* offline, once; LEMMA sidesteps a termination proof entirely, instead bounding search depth and empirically validating each benchmark path via replay. Strategy languages such as Stratego [Visser, 2001] and rewriting-logic engines such as Maude [Clavel et al., 2002] occupy the same architectural slot as LEMMA’s policy network, controlling which rule fires and in what order, but with a hand-written rather than learned strategy. Equality saturation [Tate et al., 2009, Willsey et al., 2021] takes a different approach to the same underlying search problem: rather than committing to one path, it represents many equivalent forms simultaneously via an e-graph and extracts a best one by cost function once no more rewrites apply, which sidesteps path selection at the cost of scaling with the union of all reachable forms rather than one path’s length; LEMMA instead commits to a policy-guided path, the more natural choice when the goal is one specific target expression rather than an unconstrained “best” form. Nandi et al. [2021] uses equality saturation to *synthesize* a rewrite-rule set automatically, a different point on the “what is learned” axis than LEMMA, whose seventeen template families are hand-authored and only ever selected among.

Neural-guided theorem proving. AlphaZero [Silver et al., 2018] established self-play MCTS guided by a joint policy-value network as a general search recipe; LEMMA’s search layer descends from this design, adapted to verified rewriting rather than a two-player game. GPT-f [Polu and Sutskever, 2020] and HyperTree Proof Search [Lample et al., 2022] apply this family of ideas to Metamath, Lean, and a purpose-built equation environment via online training from the prover’s own search traces. HTPS reports the same qualitative pattern this paper does: trained guidance separates solving from not solving at depth while search alone contributes little. AlphaProof [Hubert et al., 2025] scales the recipe to olympiad mathematics via large-scale auto-formalization and test-time reinforcement learning. A parallel, tactic-level branch of this literature operates over an interactive theorem prover’s full proof state and its accessible theorem library rather than a single expression: TacticToe [Gauthier et al., 2021] and TacticZero [Wu et al., 2021] learn tactic selection in HOL4, the former from human proof corpora with MCTS and the latter tabula rasa by reinforcement learning with explicit backtracking; HOList and DeepHOL-Zero [Bansal et al., 2019a,b] search a large, open premise library in HOL Light; CoqGym [Yang and Deng, 2019], PACT [Han et al., 2021], Thor [Jiang et al., 2022], LeanDojo [Yang et al., 2023], Baldur [First et al., 2023], and LISA [Jiang et al., 2021] span Coq, Lean, and Isabelle with tactics ranging from single steps to whole-proof generation. In every case, the action is a tactic call, often itself a sub-search or a call to an external hammer, over an open theorem library, and correctness is checked by a general-purpose prover’s trusted kernel; LEMMA instead selects among a small, guardrail-filtered set of rewrites on one expression, checked by a bespoke evidence-typed verifier, which is what makes a controlled, matched-pairs ablation of a single architectural choice (goal-conditioning, Section 8.3) tractable in a way it is not on top of an open-ended proof library.

Certifying algorithms and skeptical computation. LEMMA’s guardrail-and-verifier split is an instance of a much older proposer/checker separation. Result checking [Blum and Kannan, 1995] and certifying algorithms [McConnell et al., 2011] formalize a cheap, independent checker that certifies an output without trusting the computation that produced it; Alkassar et al. [2014] goes further and machine-verifies, in Isabelle/HOL, that such a checker is itself correct. This is a strictly stronger guarantee than LEMMA’s verifier, whose symbolic checks rest on a trusted but unverified implementation and whose numerical checks are explicitly non-exhaustive sampling (Table 3). Harrison and Théry [1998] pairs Maple (search) with HOL (independent re-derivation) in what is architecturally the closest historical analogue to LEMMA’s own split, decades before any learned component; Kaliszyk and Wiedijk [2007] and Théry [2001] instead build correctness

in *by construction* inside a proof assistant, the opposite trust model from LEMMA’s check-after-proposal. Harrison [2008] supplies the vocabulary this paper’s own distinction between ”traceability of evidence” and ”a proof certificate” (Section 3.1) draws on directly.

Neuro-symbolic mathematical systems. Concurrent with this work, Xie [2026] proposes a neuro-symbolic architecture pairing a hypergraph-based symbolic reasoning kernel with a transformer policy under an energy-guided objective, searched with MCTS and an evolutionary variant; the overall shape, a verified symbolic core beneath a learned policy beneath search, is the one LEMMA adopts, and this paper’s contribution is empirical rather than architectural: a controlled measurement of what that learned policy actually needs to see. AlphaGeometry [Trinh et al., 2024] pairs a language model with a symbolic deduction engine the model calls on when deduction alone stalls, verified by exact symbolic deduction throughout, on a 30-problem olympiad-geometry benchmark. FunSearch and AlphaEvolve [Romera-Paredes et al., 2024, Novikov et al., 2025] evolve candidate programs with an LLM as mutation operator and a deterministic evaluator as the only correctness signal, an evolutionary loop with no tree search or learned value estimate, targeting open-ended constructions rather than one fixed goal expression. Peano [Poesia and Goodman, 2023] defines a dependently-typed environment with a finite legal-action set at each state, close in spirit to LEMMA’s guardrail, but verifies by type-checking over a general proof system rather than symbolic-or-sampled-numerical evidence over a fixed rule vocabulary. Qin et al. [2021] injects symbolic constraints as differentiable auxiliary losses for math-word-problem solving, a ”symbolic” component with no independent, non-differentiable checker analogous to LEMMA’s verifier.

Tables 1 and 2 summarize where LEMMA sits relative to the named systems discussed above, by learned guidance, search procedure, and verification.

System	Learned guidance	Search	Verification
SymPy [Meurer et al., 2017]	none	none (fixed procedure)	none (trusted implementation)
Equality saturation [Tate et al., 2009, Willsey et al., 2021]	none	e-graph saturation, no path search	none (extraction by cost function)
AlphaZero [Silver et al., 2018]	policy + value network	MCTS (self-play)	game rules, not a proof kernel
GPT-f [Polu and Sutskever, 2020]	tactic-prediction policy	best-first proof search	Metamath kernel
HTPS [Lample et al., 2022]	policy + value network	MCTS (hypertree), on-line training	Lean / Metamath / Equations kernel
AlphaProof [Hubert et al., 2025]	policy + value network, test-time RL	MCTS	Lean kernel
TacticToe [Gauthier et al., 2021]	tactic-selection policy (from human proofs)	MCTS, open tactic library	HOL4 kernel
TacticZero [Wu et al., 2021]	tactic policy, tabula rasa RL	RL rollout with explicit backtracking	HOL4 kernel
LeanDojo / ReProver [Yang et al., 2023]	retrieval-augmented tactic policy	best-first search over ~100K-theorem library	Lean kernel

Table 1. Symbolic-rewriting and theorem-proving systems, one row per named system rather than merged by family, since kernel and training source differ across them even within a family. Target-conditioning and evidence granularity are discussed in prose above; every system here is target-conditioned (a specific theorem statement) except the first two.

System	Learned guidance	Search	Verification
AlphaGeometry [Trinh et al., 2024]	language model (auxiliary constructions)	symbolic deduction engine, not MCTS	exact symbolic deduction
FunSearch / AlphaEvolve [Romera-Paredes et al., 2024, Novikov et al., 2025]	LLM as mutation operator	evolutionary loop, no tree search	deterministic evaluator
Peano [Poesia and Goodman, 2023]	policy over typed actions	search over dependent-type environment	type-checking
Mathesis [Xie, 2026]	energy-guided transformer policy	MCTS + evolutionary variant	symbolic reasoning kernel
LEMMA (this paper)	goal-conditioned policy + value network	MCTS (PUCT)	bespoke evidence-typed verifier (Table 3)

Table 2. Neuro-symbolic, evolutionary, and typed-environment systems, and where LEMMA sits among them. FunSearch/AlphaEvolve target open-ended constructions rather than a fixed goal; the other four, including LEMMA, condition on a specific target.

3 System: The Symbolic Kernel

Every rewrite LEMMA performs passes through three layers before it is accepted, shown in Figure 1. Given the current expression, a lightweight *guardrail* first restricts the set of rules worth attempting: it profiles the expression’s shape and returns only the rules whose preconditions could plausibly apply, which keeps the branching factor the search actually faces close to the branching factor a human would recognize, rather than the full library of several hundred registered rules. The *rule engine* then applies each guardrail-admitted rule and produces candidate successor expressions. Finally, an *evidence-typed verifier* checks every candidate: symbolic equivalence where a closed-form check exists, and otherwise numerical evaluation at multiple sampled points under a seeded random generator, so that a candidate step is either accepted with an explicit type of evidence attached or rejected outright. A step that is not verifier-accepted is never presented to the search layer as an option, regardless of what the rule engine produced.

More formally, let $\mathcal{G}(s)$ be the actions admitted by the guardrail at state s , let $T_a(s)$ be the successor proposed by action a , and let $V(s, T_a(s))$ denote the verifier’s result. Search receives only the following finite action set:

$$\mathcal{A}(s) = \{a \in \mathcal{G}(s) : T_a(s) \neq s \wedge V(s, T_a(s)) \in \{\text{SYMBOLIC}, \text{NUMERIC}, \text{REPLAY}\}\}. \quad (3.1)$$

The non-identity condition prevents a rule from creating a vacuous self-loop. Equation (3.1) is the boundary between symbolic execution and learned search: the network ranks admissible actions, but it cannot make an inadmissible one available.

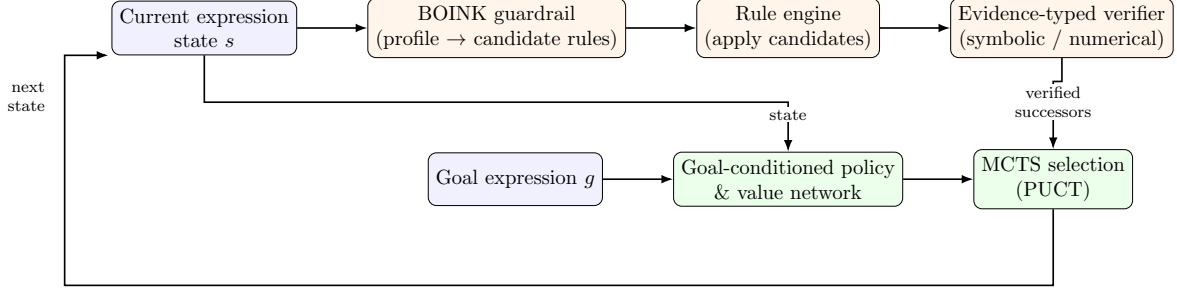


Figure 1. One iteration of LEMMA’s search loop. The guardrail, rule engine, and verifier form a trusted pipeline that only ever exposes verified successor states to the search layer; the policy and value network is conditioned on both the current state and the fixed goal expression, and its output only ranks moves the verifier has already accepted.

This ordering is deliberate: the guardrail is a performance filter, not a correctness gate, so the verifier is the last word on whether a step is legitimate, and the policy network never gets an opportunity to rank a move the verifier would have rejected. Training data generation reuses this exact pipeline. The same guardrail-then-verifier filter used at inference time enumerates legal actions during data generation, so the model is never trained on moves the real search could not itself take.

3.1 Evidence status

”Verified” is not one thing in LEMMA; it is one of five whole-trace statuses, each built from per-step evidence of one of three strengths (Table 3). A step is checked by *symbolic equivalence* when both sides reduce to the same canonical form, the strongest check available, though still an equivalence check rather than a machine-checked soundness proof of the rule itself. Where no closed-form check applies, the verifier falls back to *numeric sampling*: both sides are evaluated at multiple points under a seeded generator and compared, which is evidence, not proof, since sampling can miss a disagreement at an unsampled point. A third, weaker method, *rule replay only*, applies where the expression contains a derivative or integral the evaluator cannot sample; it confirms the step reproduced the rule’s own claimed output, not that the rule is sound. Figure 2 shows how a single proposed step is classified, and how per-step evidence combines into the whole-trace status this paper reports for a solution.

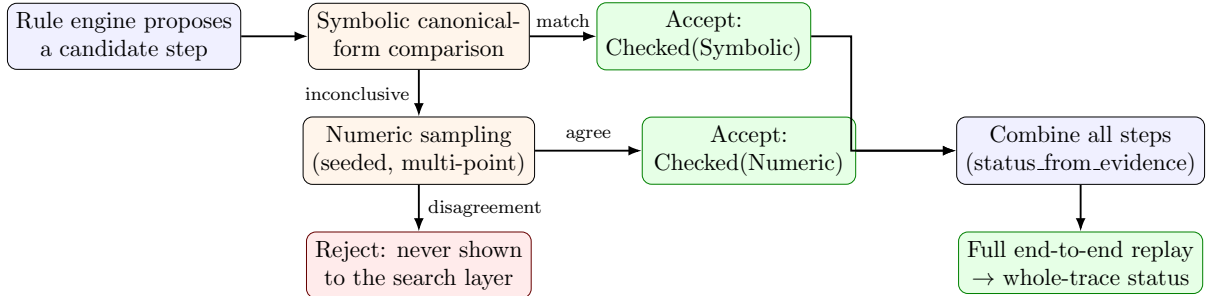


Figure 2. Trust boundary for one proposed step, and how per-step evidence becomes the whole-trace status of Table 3. A rejected step never reaches the search layer as an option (Section 3); an accepted trace is only reported as a solution after a full end-to-end replay from the original input to the final output.

Level	Status	Acceptance semantics
per-step	Symbolic equivalence	Both sides reduce to the same canonical form.
	Numeric sampling	Both sides agree at every sampled point; evidence, not proof.
	Rule replay only	Re-running the rule reproduced its claimed output; nothing further checked (derivatives/integrals).
whole-trace	Checked	Replays end to end; every step independently checked (symbolic or numeric), none by replay-only.
	Partial	Replays; at least one step is weaker than the rest (a replay-only step, or a mix of symbolic and numeric).
	Heuristic	Replays; every step relies on the weaker methods throughout (all numeric, or all replay-only).
	Unverified	Does not replay end to end, or contains an unchecked step.
	Unsupported	The requested verification mode is not implemented for this case.

Table 3. The evidence taxonomy LEMMA actually implements (`mm-verifier::status`). Only **Checked** is reported as fully verified; every other status is retained and shown, not silently upgraded. All results in Section 8 come from replayed, **Checked** or **Partial** traces; the corpora never contain a derivative or integral, so **Heuristic** via replay-only does not arise in this paper’s experiments.

4 The Neural Policy and Value Network

The network is a small transformer (embedding dimension 256, hidden dimension 512, 8 attention heads, 4 layers) with a token vocabulary of 64 symbols. It is sized for this task rather than adapted from a general-purpose language model. A recursive-descent tokenizer maps the current state s and target g to the goal-conditioned sequence

$$x(s, g) = [\text{START}] \text{ tok}(s) [\text{SEP}] \text{ tok}(g) [\text{END}]. \quad (4.1)$$

The separator reuses a vocabulary slot that earlier versions of the encoder left unassigned. No tensor shape changed when goal conditioning was added, so previously trained state-only checkpoints remain loadable. Figure 3 shows the complete forward pass.

For a padded batch, $m_i \in \{0, 1\}$ marks a real token. If $H^{(0)} = E(x) + P$ is the token-plus-position representation and F_ℓ is transformer block ℓ , the pooled representation is

$$H^{(\ell+1)} = F_\ell(H^{(\ell)}; m), \quad z = \frac{\sum_{i=1}^L m_i H_i^{(4)}}{\sum_{i=1}^L m_i}. \quad (4.2)$$

The same mask is added to the attention scores before softmax and used in the pooling denominator. Thus padded positions receive neither attention mass nor weight in z .

The policy head produces logits $\ell_a(s, g)$ for 572 rule actions plus a stop class. The value head returns $\hat{v}_\theta(s, g) = \tanh(w_v^\top z + b_v) \in [-1, 1]$. At both training and inference, the action distribution is normalized only over the verifier-accepted set from Equation (3.1):

$$\pi_\theta(a \mid s, g) = \begin{cases} \frac{\exp(\ell_a(s, g))}{\sum_{b \in \mathcal{A}(s)} \exp(\ell_b(s, g))}, & a \in \mathcal{A}(s), \\ 0, & a \notin \mathcal{A}(s). \end{cases} \quad (4.3)$$

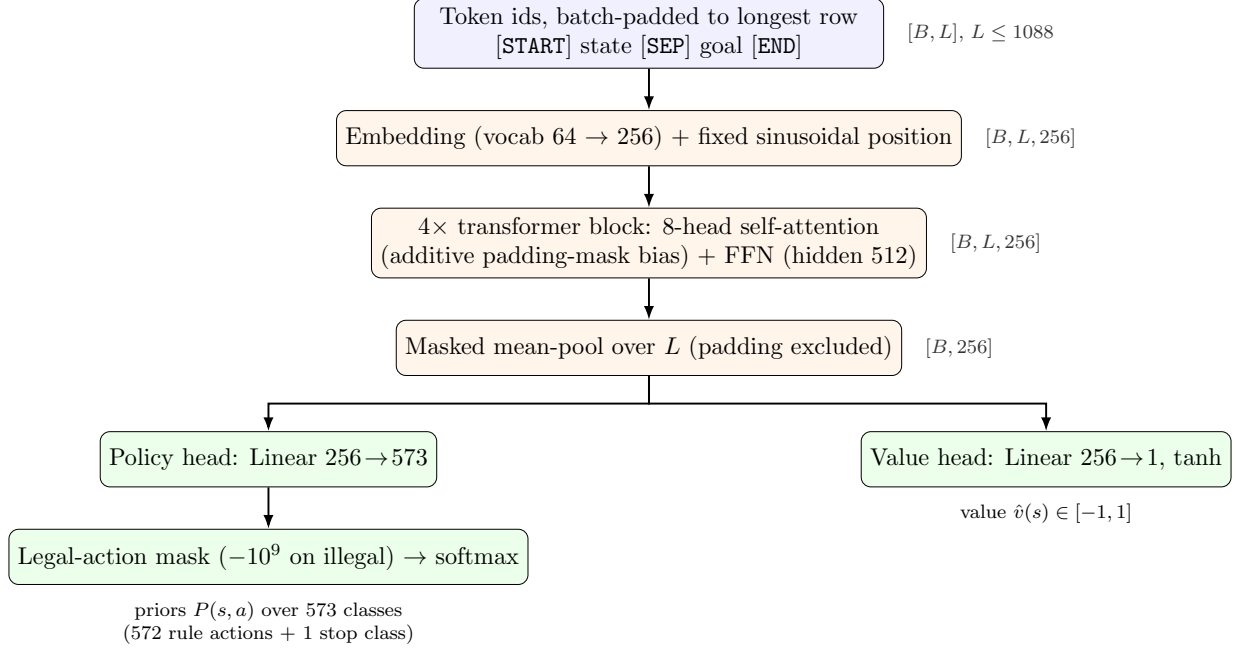


Figure 3. The full forward pass, with tensor shapes (B = batch, L = sequence length). Both heads share every layer up to the pooled representation; the legal-action mask is applied identically at training time (inside the cross-entropy loss, Section 5) and at inference time (before PUCT reads the priors, Section 6), so the network never has an opportunity to prefer an action the symbolic pipeline has not already certified as legal.

The encoder accepts each expression at its natural token length, or returns a typed error when the configured bound is exceeded. This matters for the long-horizon corpus: depth-8 inputs require 45 tokens, while depth-128 inputs require 531–615. Batches pad only to their longest row. An additive attention bias and a masked pooling denominator ensure that padding contributes neither to attention nor to the pooled representation; a regression test compares tight and padded encodings of the same expression and requires near-identical output.

A model’s manifest records a single boolean, `goal_conditioned`, and the search layer consults it before every policy or value query: a goal-conditioned model is never fed a state-only encoding, and a state-only model is never asked to condition on a goal it was not trained to use. This is enforced in code, not by convention, specifically so that the ablation in Section 8.3 cannot be silently invalidated by the two arms being evaluated inconsistently.

5 Training Procedure

Training examples are generated from a set of seventeen parametrized “core” templates (Table 4): collecting like terms, constant folding, distributing a product over a sum, the difference-of-squares identity, zero-annihilation, the Pythagorean identity, sine-arcsine and cosine-arccosine cancellation, six families of equation-preserving cancellation, and three families in which only the goal (not the state) determines the correct move. Each template specifies a genuinely solvable start-to-goal pair together with the exact rule sequence connecting them. Table 4 lists every core family; four of the seventeen (`collect-like-terms`, `const-fold`, `distribute`, `zero-annihilation`) are used as the innermost solvable kernels in the benchmark (Section 7). Several rules from other families, including `power_of_one`, `identity_mul_one`, `identity_add_zero`, and `double_negative`, appear

only as *wrapper* motifs around those cores (Table 5); the remaining thirteen families are exercised only during training.

Core family	Representative rewrite	Rule key	In benchmark
collect-like-terms	$ax + bx \rightarrow (a+b)x$	algebra::collect_like_terms	as core
const-fold	$a + b \rightarrow (a+b)$	algebra::const_fold	as core
distribute	$a(x+b) \rightarrow ax + ab$	algebra::distribute	as core
difference-of-squares	$x^2 - b^2 \rightarrow (x+b)(x-b)$	algebra::difference_of_squares	no
zero-annihilation	$0 \cdot (ay + x) \rightarrow 0$	algebra::zero_mul	as core
goal-decides-distribute	$1 \cdot (P+0) \rightarrow \text{state } or \text{ distributed, by goal}$	algebra::distribute	no
goal-decides-power-one	$((x+a)^2)^1 \rightarrow (x+a)^2$, if goal selects it	algebra::power_of_one	as wrapper only
goal-decides-power-mul	$((x+a)^2)^1 \rightarrow (x+a)^{2 \cdot 1}$, if goal selects it	algebra::power_mul	no
pythagorean-identity	$\sin^2(x+a) + \cos^2(x+a) \rightarrow 1$	trig::pythagorean_identity	no
sin-arcsin	$\sin(\arcsin(\sin x)) \rightarrow \sin x$	trig::sin_arcsin	no
cos-arccos	$\cos(\arccos(\cos y)) \rightarrow \cos y$	trig::cos_arccos	no
equation-cancel-addition	$x + a = b \rightarrow x = b - a$	equations::cancel_addition	no
equation-cancel-subtraction	$x - a = b \rightarrow x = b + a$	equations::cancel_subtraction	no
equation-cancel-multiplication	$ax = b \rightarrow x = b/a$	equations::cancel_multiplication	no
equation-cancel-division	$x/a = b \rightarrow x = ab$	equations::cancel_division	no
equation-linear-solve	$ax + b = c \rightarrow x = (c-b)/a$	equations::linear_solve	no
equation-mixed-cancellation	nested affine chain $\rightarrow x = \text{solution}$	equations::cancel.* (mixed)	no

Table 4. Every core family in the training generator (`data_deep.rs`). "In benchmark" marks whether the family's core problem (not merely one of its rules) appears in the frozen evaluation corpus; `power_of_one` is the one case where a family's rule appears in the benchmark only as a wrapper around a different core, not as its own core problem.

Long problems are built by wrapping a core in "collision layers" (an identity transformation such as $1 \cdot (P + 0)$, at which both the intended peeling move and an unrelated distribution move are simultaneously legal and verifier-accepted) and other one-step motifs, chosen so that the same three or four surface shapes cannot be memorized as a fixed prefix. Every candidate long problem is replayed against the real rule engine and verifier before being accepted; a construction that does not replay exactly onto its stated goal is discarded rather than relabeled. Figure 4 traces this construction end to end, from one core template to the training rows a model actually optimizes against.

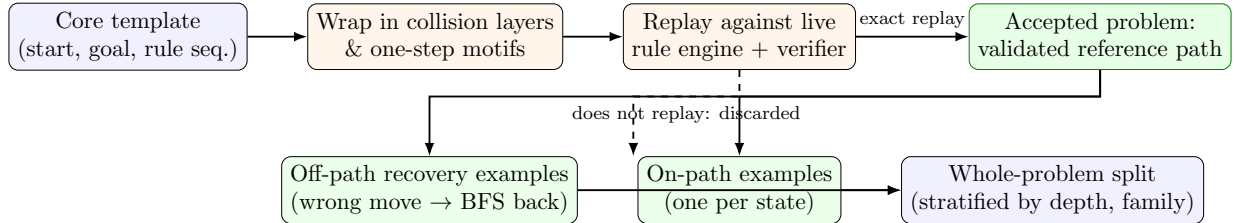


Figure 4. Training-data construction. A construction that fails to replay exactly onto its stated goal is discarded, not relabeled (Section 5); the whole-problem, depth-and-family-stratified split at the end is what makes v1/v2/v3's held-out validation whole-problem rather than whole-state (Appendix A discusses what this split does and does not test).

One family, `goal-decides-distribute`, is included specifically because without it a policy can score well everywhere else while never actually consulting the goal: at the shape $1 \cdot (P + 0)$, every other family in the corpus answers "peel the identity," so a model can learn a shape-to-label mapping that ignores the goal entirely and still appear goal-conditioned by the encoding alone. `goal-decides-distribute` presents the identical shape with the opposite correct answer, forcing the label to depend on the goal rather than the state. We verify this property directly with a regression test before relying on it in the ablation.

Each training example has a verified legal-action set $\mathcal{A}(s)$ and a nonempty subset $\mathcal{R}(s, g) \subseteq \mathcal{A}(s)$ of actions that reach the reference successor. When several actions yield the same correct successor, the target does not arbitrarily choose one. It assigns them equal probability:

$$q(a \mid s, g) = \begin{cases} |\mathcal{R}(s, g)|^{-1}, & a \in \mathcal{R}(s, g), \\ 0, & \text{otherwise.} \end{cases} \quad y(r) = 2 \cdot 0.97^r - 1, \quad (5.1)$$

where r is the number of reference steps remaining. The value target $y(r)$ gives the value head a monotone signal rather than a constant target.

The implemented objective combines legal-set cross-entropy with value regression, using the configured value weight $\lambda = 0.5$:

$$\mathcal{L}(\theta) = - \sum_{a \in \mathcal{A}(s)} q(a \mid s, g) \log \pi_{\theta}(a \mid s, g) + \lambda (\hat{v}_{\theta}(s, g) - y(r))^2. \quad (5.2)$$

Illegal logits receive a -10^9 mask before the log-softmax. They therefore receive neither policy probability nor gradient through the policy objective.

Some training runs additionally include off-path recovery data. From an on-path state, we take a legal but incorrect move and run bounded breadth-first search for a short return to the reference path. The policy therefore sees some examples of recovery after a wrong turn rather than only states on a correct solution. The recovery share is 13.4% for v1, 3.5% for v2, and 0% for v3 and both ablation arms (Appendix A).

Training uses AdamW, a cosine learning-rate schedule with warmup, and early stopping on a held-out whole-problem split stratified by depth and family. Each epoch’s weights are saved to the primary output path; the released checkpoint is therefore the last epoch completed by that run. Appendix B records this selection rule alongside each arm’s observed training duration.

5.1 Seeded training protocol

Each training invocation receives an explicit `--seed`. The implementation reseeds the device generator and derives a separate seed for per-epoch batch ordering, while holding the generated data and its train/validation split fixed across a seed-matched comparison. A seed sweep therefore changes weight initialization and optimization trajectory, not the examples assigned to training or validation. Section 8.3 uses this protocol for the paired target-conditioning study.

6 Guided Search

Search is standard PUCT-style MCTS [Silver et al., 2018], adapted so that a node’s children are exactly the guardrail-filtered, verifier-accepted successors described in Section 3. It can never expand into a state the trusted pipeline would not itself certify. At a state s , selection chooses

$$a^* = \arg \max_{a \in \mathcal{A}(s)} \left[Q(s, a) + c_{\text{puct}} \pi_{\theta}(a \mid s, g) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right]. \quad (6.1)$$

Here $Q(s, a)$ is the running mean of backed-up values, $N(s)$ is the parent visit count, and $N(s, a)$ is the edge visit count. The implementation uses $c_{\text{puct}} = 1.41$ (Appendix B).

A newly reached leaf is expanded, evaluated by the value network, and backed up along the selected path. Goal states contribute a terminal value of 1; nonterminal leaves contribute $\hat{v}_{\theta}(s, g)$.

When the model is goal-conditioned, every policy and value query includes the fixed target expression. State-only models use the corresponding state-only calls, selected by the manifest flag from Section 4. Search stops on reaching the goal or exhausting its simulation budget, then returns the best path found and the number of expanded nodes.

Figure 5 shows one real, measured decision point rather than a schematic one. At step 32 of benchmark problem **deep-00D-d64-001**, the current state has the shape $(\dots)^2$ raised to the power 1, at which two rules are simultaneously legal and verifier-accepted: **power_of_one** (peel the outer power, the reference-correct move) and **power_mul** (multiply the two exponents instead, a legal but incorrect move, the **power_of_one**-family distractor in Table 5). We queried model v3 (Appendix A) at this exact state and goal; its raw priors are 0.0354 for **power_of_one** against 6×10^{-8} for **power_mul**, renormalized to 100.0% of the legal mass on the correct move. PUCT selects **power_of_one**; the verifier confirms the resulting step by symbolic canonical-form comparison (Section 3), the case for a definitional identity like this one, and search continues from the peeled state.

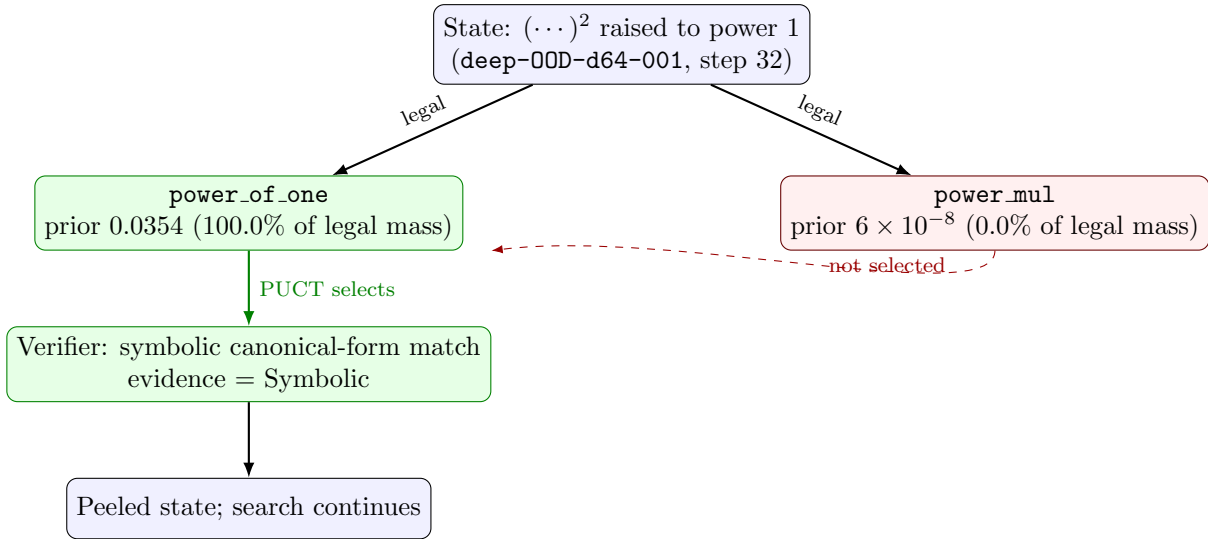


Figure 5. A real, measured decision point: both rules are legal and verifier-accepted at this state, but the goal-conditioned policy’s priors concentrate almost all mass on the reference-correct move (raw values from model v3, not illustrative numbers). This is the mechanism Table 5’s **power_of_one** branching statistics describe in aggregate: a distractor is present, and getting it wrong here loses the problem, since a depth-64 path offers roughly 64 such choices in a row.

7 Experimental Setup

Benchmark corpus. We designed a frozen, expert-reviewed benchmark of 120 long-horizon rewriting problems at depths $\{64, 96, 128\}$, with twenty problems per (split, depth) cell. The **deep-ID** split wraps a solvable core in identity collision layers; the **deep-00D** split uses one of two fixed wrapper motifs per problem: repeated **power_of_one** steps or repeated **double_negative** steps. Exact-input collisions are excluded. Every reference path is also validated end to end by parsing, replaying it against the live rule engine and verifier, and checking an exact round trip. Validation confirms the intended number of problems per cell and the absence of duplicate (input, target) pairs.

Split	Wrapper family	Problems	States	Mean branch.	Max branch.	Decision points
deep-ID	identity	60	5,760	1.49	2	2,832 (49%)
deep-OOD	double_negative	31	2,848	1.00	2	12 (0.4%)
deep-OOD	power_of_one	29	2,912	1.98	2	2,854 (98%)

Table 5. Verified branching factor at every state of every reference path in the evaluation corpus, measured directly rather than assumed. `double_negative` is a forced single-path corridor and is excluded from the 89-problem "meaningful" subset used throughout Section 8; `identity` and `power_of_one` are not.

Why a 120-problem subset, and why 89 of it. We additionally measured the verified branching factor at every state of every reference path with a standalone tool that counts children exactly the way the search does (guardrail, applicability, must-change-state, verifier-accepted), rather than estimating it. Table 5 shows why this matters: `double_negative` states offer essentially one legal move (mean branching 1.00, a decision point at only 12 of 2,848 states), so any method solves this subset by simply always being able to walk the one path forward.

The `power_of_one` family, by contrast, offers a real distractor at 98% of states (mean branching 1.98), so a depth-128 problem in this family hides its solution among roughly 2^{128} candidate paths. Reporting one number for a benchmark split that mixes a free 31-problem subset with a 29-problem subset not solved by unguided search at the evaluated budget (`identity` contributes a further 60 non-forced problems) obscures which of the two is actually being measured; we therefore report solve rate on the 89-problem subset that excludes the forced-corridor family throughout, and note the full 120-problem totals alongside it for completeness.

Search budget. The six-way comparison uses a 400-simulation budget for MCTS and a 400 generated-successor cap for beam search; greedy decoding is bounded only by the same maximum search depth, set to the deepest benchmark problem plus 8. The beam curve in Table 7 additionally evaluates caps 800 and 1600. In every case the depth bound exceeds the reference solution length, so solution length is not itself the limiting factor.

Compute. Evaluation runs on a 16-logical-core workstation CPU; the 120 problems in a given arm are independent searches with no shared mutable state, so we parallelize across problems with a work-stealing thread pool rather than evaluating them one at a time, which we verified produces byte-identical per-problem outcomes to the original sequential implementation (same solved/unsolved status, same node counts, same per-family breakdown) at roughly seven times the throughput. Training runs on a single rented NVIDIA A10G GPU (24 GB) via Modal, using CUDA 12.4.1, Rust’s candle-core 0.8.4 as the tensor backend, and cudarc 0.13.9; exact package versions and the Modal image definition are in the released harness.

8 Results

8.1 Learned action guidance solves the controlled long-horizon corpus

Table 6 evaluates the frozen v3 checkpoint through six decision procedures over the same guardrail-filtered, verifier-accepted successors, all at a 400 generated-successor budget where applicable. Uniform search and the learned-value-only variant solve only the forced-corridor family. In contrast, learned policy guidance solves every meaningful instance under greedy decoding.

At the 400-budget comparison point, uniform search, value-only MCTS, and beam search reach only the forced-corridor `double_negative` family. The policy-only MCTS variant solves 87/89 meaningful cases, while greedy decoding solves all 89. Thus, on this controlled corpus, the learned action policy is the most budget-efficient component; the present value head does not improve over

Decision procedure	Total (/120)	identity (/60)	power_of_one (/29)	Meaningful (/89)
Uniform MCTS	31	0	0	0
Value-only MCTS	31	0	0	0
Beam search (width 8)	31	0	0	0
Policy-value MCTS	92	44	17	61
Policy-only MCTS	118	60	27	87
Greedy learned policy	120	60	29	89

Table 6. Six decision procedures using the same frozen v3 checkpoint where applicable. “Meaningful” excludes the 31 forced-corridor problems (Table 5). Greedy and beam use the same verifier-admitted successor function as MCTS; beam has width 8 and a 400 generated-successor cap.

direct policy execution. This comparison is one frozen-checkpoint study, not a claim that greedy decoding dominates search in other symbolic domains.

Beam budget accounting. The greedy and beam implementations enumerate the same verifier-admitted successors and differ only in how they retain paths. At the 400 generated-successor cap, greedy evaluates a mean 144.5 tree nodes per problem (median 144), whereas width-8 beam evaluates a mean 320.9 (median 400). Instrumented beam runs report **BudgetExhausted** for every one of the 89 decision-bearing problems, with a mean 249.4 expanded frontier states and a mean retained-path depth of 56.1; none reaches its target. Increasing the cap changes that result: beam solves 28/89 meaningful problems at 800 and all 89/89 at 1600 (Table 7). The curve shows a budget-efficiency gap, not a categorical inability of beam search to solve the corpus.

Generated-successor cap	Total (/120)	Meaningful (/89)	Mean nodes	Beam termination
400	31	0	320.9	89 budget / 31 goal
800	59	28	572.3	61 budget / 59 goal
1600	120	89	735.9	120 goal

Table 7. Width-8 beam search with the same frozen v3 policy and verifier-admitted successors as Table 6. “Budget” denotes a run that exhausted its generated-successor cap; “goal” denotes a run that reached the exact target. The 800 and 1600 runs use the released Modal A10G harness; outcome and node accounting are independent of hardware.

Value-head calibration. To interpret the policy-value result, we replayed the frozen reference paths and queried the v3 value head every eight states, retaining each final state. This yields 1,560 on-path estimates spanning all depths, plus 528 verifier-accepted non-reference successors at the sampled branching states. The value estimate has Pearson correlation 0.370 with its own monotone training target $y(r)$ and -0.398 with remaining reference steps. Its on-path and off-path distributions are nearly indistinguishable in aggregate (means 0.0559 and 0.0532, respectively; medians 0.1087 and 0.0671). These measurements do not identify one unique source of the policy-value gap, but they show that the present value signal is weakly calibrated to path progress and provides little aggregate separation between replayed and alternative successors. That is consistent with the observed ordering: policy-only MCTS solves 87/89 meaningful problems, while adding this value estimate reduces the score to 61/89.

Table 8 records the earlier policy-value MCTS training lineage by construction depth. It provides training-history context only: the controlled six-way comparison above is the paper’s evidence about the relative roles of policy, value, and search. We do not use this single-run lineage to make a causal claim about depth or training recipe.

Arm	depth 64 (/28)	depth 96 (/29)	depth 128 (/32)
Uniform prior	0	0	0
Trained, v1	19	20	20
Trained, v2	12	10	10
Trained, v3 (no recovery)	28	19	14

Table 8. Solved meaningful problems (`identity` + `power_of_one`) by construction depth. Depth cells are unequal size (28/29/32) because `double_negative` problems are excluded per cell, not because the benchmark itself is unbalanced (Table 5 confirms 20 problems per cell before that exclusion).

Arm	Seed 1	Seed 2	Seed 3	Mean	Std. dev.
With goal	89	58	88	78.3	17.6
No goal	89	68	89	82.0	12.1

Table 9. Goal-conditioning ablation, three independently seeded runs per arm, scored on the 89-problem meaningful subset. Weight initialization, batch order, and training data are the only sources of variation across seeds within an arm; training data, labels, and hyperparameters are identical between the two arms.

8.2 The v1/v2/v3 lineage

v1, v2, and v3 are three successive, single training runs that differ in more than one respect (training data breadth, encoder width, and whether off-path recovery examples were included), so we do not treat them as a matched study the way Section 8.3 is. We isolated the recovery-data variable directly by holding v2’s architecture, data, and hyperparameters fixed and setting `--recovery-per-problem 0` for v3; the result (Appendix A) is a single-run finding worth further investigation, not an established causal effect.

8.3 Goal-conditioning: a seed-matched ablation

To isolate goal-conditioning specifically, we train two models on byte-identical problems, labels, legal-action masks, random seed, network architecture, and hyperparameters, differing in exactly one input: the state-only arm’s training rows are produced by truncating the goal-conditioned arm’s already-generated token rows at the `[SEP]` boundary and re-appending `[END]`, which guarantees the two arms cannot differ in anything else. Both arms use a lighter training budget than the v1/v2/v3 lineage (fewer problems per depth, fewer epochs), chosen to make a three-seed replication affordable; we report the two arms’ scores relative to each other, not against v1/v2/v3, which used a different budget and an earlier, narrower training-data configuration.

Under the seeded protocol of Section 5.1, we train three seeds per arm, each seed value reused for both arms so the comparison is paired: seed i ’s with-goal and no-goal models share weight initialization and batch order, differing only in the goal-conditioning treatment. Table 9 and Figure 6 show the result. The paired difference (with-goal minus no-goal, out of 89) is 0, -10 , and -1 across the three seeds: in no case did removing the goal reduce solve rate.

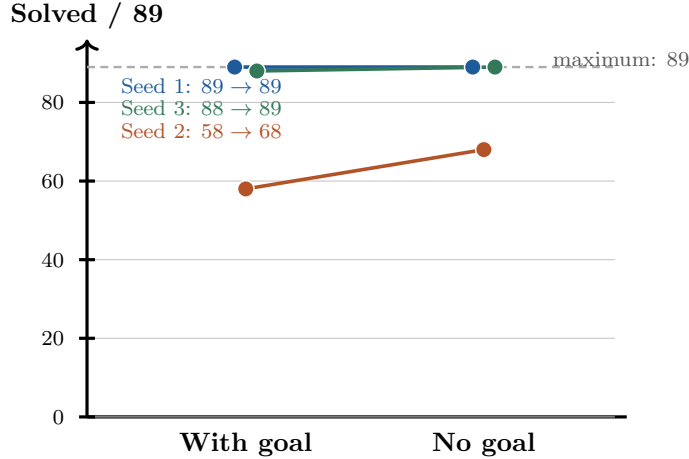


Figure 6. Paired seed comparison for the goal-conditioning ablation: each line connects one seed’s with-goal and no-goal solve count (out of 89 meaningful problems), so slope shows the within-seed effect of removing the goal rather than an across-seed average. Δ is with-goal minus no-goal. Two of three seeds show a difference of -1 or 0 ; the third shows -10 . In both cases no-goal did not score lower.

We observed no consistent improvement from goal-conditioning across three paired seeds; the experiment is underpowered to exclude a smaller effect, and a benefit such as faster convergence or better sample efficiency would not necessarily show up in a fixed-budget, fixed-epoch comparison at all. A plausible mechanism: our benchmark’s reference solutions are the *unique* correct path in an otherwise heavily disambiguated corpus (Table 5 shows only two families ever offer a real alternative at all), so a policy that has learned “prefer the unwrapping move over the multiplying move” as a general prior can match a large fraction of reference paths without consulting an explicit goal token, provided its training data teaches that prior at sufficient depth and diversity. The training corpus includes a family, **goal-decides-distribute** (Section 5), constructed so this shortcut cannot work *everywhere*; it is nonetheless sufficient to match goal-conditioned performance in aggregate on this benchmark, at this training budget. Isolating goal-conditioning’s remaining contribution, if any, needs a larger seed count or a benchmark constructed so a state-only shortcut cannot exist at all.

9 Limitations

Scope. LEMMA covers seventeen template families in algebra, trigonometry, and linear equation solving. The results do not establish transfer to other mathematical domains, natural-language reasoning, or general theorem proving.

Statistical power. The ablation has three seeds per arm. That is enough to reject the earlier single-run interpretation, but not to estimate a small effect precisely. The v1/v2/v3 lineage (Appendix A) has one run per arm and is descriptive rather than causal.

Verifier scope. Numerical verification samples a bounded number of points under a seeded generator; it is evidence, not proof, for identities the symbolic checker cannot close directly, and a pathological counterexample at unsampled points, while not observed in this work, is not excluded by construction.

Comparison scope. The benchmark compares decision procedures over a common verifier-admitted action space. It is not a cross-system leaderboard for computer algebra systems or theorem provers, which use different rule inventories and objectives.

10 Conclusion

On the frozen 89-problem decision-bearing benchmark, learned action guidance is sufficient for complete solution under greedy decoding. Uniform MCTS and value-only MCTS solve none of these problems; policy-only MCTS solves 87 and policy-value MCTS solves 61. Width-8 beam search rises from 0/89 at a 400 generated-successor cap to 28/89 at 800 and 89/89 at 1600. The controlled comparison therefore identifies the learned action policy as the most budget-efficient component in this setting: greedy decoding reaches the full corpus without tree search, while the present value head does not improve on it and beam requires a larger budget. The paired three-seed target-conditioning study finds no measured improvement from explicit target input under its evaluated protocol. Broader target-dependent and external evaluations remain important next tests of the same architecture.

References

- A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, Š. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, A. Scopatz. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017.
- D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.
- S. Polu, I. Sutskever. Generative Language Modeling for Automated Theorem Proving. *arXiv:2009.03393*, 2020.
- G. Lample, T. Lacroix, M.-A. Lachaux, A. Rodriguez, A. Hayat, T. Lavril, G. Ebner, X. Martinet. HyperTree Proof Search for Neural Theorem Proving. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, P. Panchekha. egg: Fast and Extensible Equality Saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL), 2021.
- T. Hubert, R. Mehta, L. Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025.
- K. Xie. Constructing a Neuro-Symbolic Mathematician from First Principles. *arXiv:2601.00125*, 2026.
- G. Lample, F. Charton. Deep Learning for Symbolic Mathematics. *International Conference on Learning Representations (ICLR)*, 2020. *arXiv:1912.01412*.
- D. Saxton, E. Grefenstette, F. Hill, P. Kohli. Analysing Mathematical Reasoning Abilities of Neural Models. *arXiv:1904.01557*, 2019.
- F. Charton. Linear Algebra with Transformers. *Transactions on Machine Learning Research*, 2022. *arXiv:2112.01898*.

- S. d’Ascoli, P.-A. Kamienny, G. Lample, F. Charton. Deep Symbolic Regression for Recurrent Sequences. *International Conference on Machine Learning (ICML)*, 2022. *arXiv:2201.04600*.
- I. Drori, S. Zhang, R. Shuttlesworth, L. Tang, A. Lu, E. Ke, K. Liu, L. Chen, S. Tran, N. Cheng, R. Wang, N. Singh, T. L. Patti, J. Lynch, A. Shporer, N. Verma, E. Wu, G. Strang. A Neural Network Solves, Explains, and Generates University Math Problems by Program Synthesis and Few-Shot Learning at Human Level. *Proceedings of the National Academy of Sciences*, 119(32):e2123433119, 2022. *arXiv:2112.15594*.
- A. Davies, P. Veličković, L. Buesing, S. Blackwell, D. Zheng, N. Tomašev, R. Tanburn, P. Battaglia, C. Blundell, A. Juhász, M. Lackenby, G. Williamson, D. Hassabis, P. Kohli. Advancing Mathematics by Guiding Human Intuition with AI. *Nature*, 600:70–74, 2021.
- D. E. Knuth, P. B. Bendix. Simple Word Problems in Universal Algebra. In J. Leech (ed.), *Computational Problems in Abstract Algebra*, pp. 263–297. Pergamon Press, 1970.
- N. Dershowitz. Termination of Rewriting. *Journal of Symbolic Computation*, 3(1–2):69–116, 1987.
- F. Baader, T. Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- M. Bezem, J. W. Klop, R. de Vrijer, eds. (“Terese”). *Term Rewriting Systems*. Cambridge Tracts in Theoretical Computer Science 55. Cambridge University Press, 2003.
- E. Visser. Stratego: A Language for Program Transformation Based on Rewriting Strategies (System Description of Stratego 0.5). *Rewriting Techniques and Applications (RTA)*, LNCS 2051, pp. 357–362. Springer, 2001.
- M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Oliet, J. Meseguer, J. F. Quesada. Maude: Specification and Programming in Rewriting Logic. *Theoretical Computer Science*, 285(2):187–243, 2002.
- R. Tate, M. Stepp, Z. Tatlock, S. Lerner. Equality Saturation: A New Approach to Optimization. *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 264–276, 2009.
- C. Nandi, M. Willsey, A. Zhu, Y. R. Wang, B. Saiki, A. Anderson, A. Schulz, D. Grossman, Z. Tatlock. Rewrite Rule Inference Using Equality Saturation. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA), 2021. *arXiv:2108.10436*.
- T. Gauthier, C. Kaliszyk, J. Urban, R. Kumar, M. Norrish. TacticToe: Learning to Prove with Tactics. *Journal of Automated Reasoning*, 65(2):257–286, 2021. *arXiv:1804.00596*.
- K. Bansal, S. M. Loos, M. N. Rabe, C. Szegedy, S. Wilcox. HOList: An Environment for Machine Learning of Higher-Order Theorem Proving. *International Conference on Machine Learning (ICML)*, 2019. *arXiv:1904.03241*.
- K. Bansal, C. Szegedy, M. N. Rabe, S. M. Loos, V. Toman. Learning to Reason in Large Theories without Imitation. *arXiv:1905.10501*, 2019.
- K. Yang, J. Deng. Learning to Prove Theorems via Interacting with Proof Assistants. *International Conference on Machine Learning (ICML)*, 2019. *arXiv:1905.09381*.

- J. M. Han, J. Rute, Y. Wu, E. W. Ayers, S. Polu. Proof Artifact Co-training for Theorem Proving with Language Models. *International Conference on Learning Representations (ICLR)*, 2021. *arXiv:2102.06203*.
- M. Wu, M. Norrish, C. Walder, A. Dezfouli. TacticZero: Learning to Prove Theorems from Scratch with Deep Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. *arXiv:2102.09756*.
- A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygóźdź, P. Miłoś, Y. Wu, M. Jamnik. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. *arXiv:2205.10893*, 2022.
- K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, A. Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. *arXiv:2306.15626*.
- E. First, M. N. Rabe, T. Ringer, Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023. *arXiv:2303.04910*.
- A. Q. Jiang, W. Li, J. M. Han, Y. Wu. LISA: Language Models of Isabelle Proofs. *6th Conference on Artificial Intelligence and Theorem Proving (AITP)*, 2021.
- M. Blum, S. Kannan. Designing Programs that Check Their Work. *Journal of the ACM*, 42(1):269–291, 1995.
- R. M. McConnell, K. Mehlhorn, S. Näher, P. Schweitzer. Certifying Algorithms. *Computer Science Review*, 5(2):119–161, 2011.
- E. Alkassar, S. Böhme, K. Mehlhorn, C. Rizkallah. A Framework for the Verification of Certifying Computations. *Journal of Automated Reasoning*, 52(3):241–273, 2014. *arXiv:1301.7462*.
- J. Harrison, L. Théry. A Skeptic’s Approach to Combining HOL and Maple. *Journal of Automated Reasoning*, 21(3):279–294, 1998.
- C. Kaliszyk, F. Wiedijk. Certified Computer Algebra on Top of an Interactive Theorem Prover. *Towards Mechanized Mathematical Assistants (MKM/Calculemus)*, LNCS 4573, pp. 94–105. Springer, 2007.
- L. Théry. A Machine-Checked Implementation of Buchberger’s Algorithm. *Journal of Automated Reasoning*, 26(2):107–137, 2001.
- J. Harrison. Formal Proof: Theory and Practice. *Notices of the American Mathematical Society*, 55(11):1395–1406, 2008.
- T. H. Trinh, Y. Wu, Q. V. Le, H. He, T. Luong. Solving Olympiad Geometry without Human Demonstrations. *Nature*, 625:476–482, 2024.
- B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, A. Fawzi. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625:468–475, 2024.

- A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv:2506.13131*, 2025.
- G. Poesia, N. D. Goodman. Peano: Learning Formal Mathematical Reasoning. *Philosophical Transactions of the Royal Society A*, 381(2251), 2023. *arXiv:2211.15864*.
- J. Qin, X. Liang, Y. Hong, J. Tang, L. Lin. Neural-Symbolic Solver for Math Word Problems with Auxiliary Tasks. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, pp. 5870–5881, 2021. *arXiv:2107.01431*.

A The v1/v2/v3 Lineage

v1, v2, and v3 differ in more than one respect, including training data breadth, encoder width, and whether off-path recovery examples were included. Each is a single training run, so we do not treat them as a matched study the way Section 8.3 is. We isolated the recovery-data variable directly: v3 is v2’s exact architecture, training data, and hyperparameters with `--recovery-per-problem 0`, and its 89-problem score (61) is close to v1’s (59) and well above v2’s (32), the run that included recovery data. We report this as a genuine, single-run finding worth further investigation. In this particular training configuration, recovery data correlated with lower rather than higher solve rate, and we explicitly do not claim it as an established causal effect, since establishing that would need the same seed-matched treatment Section 8.3 gives goal-conditioning.

A.1 Why v2’s validation accuracy did not predict its solve rate

v2 reached 100% legal-action top-1 accuracy on its own held-out validation split by epoch 1, yet solved only 32 of 89 meaningful benchmark problems (Table 6). This is the largest validation/benchmark gap of any arm in this paper, and one worth explaining rather than reporting as a curiosity. We measured v2’s actual per-state top-1 accuracy directly on the frozen benchmark corpus (not its own training-distribution validation split) by replaying each benchmark problem’s reference path and checking whether the policy’s top legal-action prediction matches the reference rule at every state, on small stratified samples (11 `identity` problems, 1,344 states; 2 `power_of_one` problems, 128 states) chosen to avoid contending for CPU with the budget-curve experiment (Appendix C) running concurrently.

The result does not have one clean explanation. For `power_of_one`, measured per-state accuracy is 98.39%, but naive compounding (p^{depth}) already predicts only 35%/21%/13% success at depths 64/96/128, which is consistent with the observed collapse to 0/29, though the true failure is total while the naive arithmetic is not, plausibly because MCTS must also *find* the correct branch at a near-50/50 decision (Table 5) at every one of up to 128 levels, a second bottleneck beyond raw policy accuracy. For `identity`, this explanation fails outright: measured per-state accuracy is 100% in every one of the 1,344 sampled states, yet only 32 of 60 `identity` problems (53%) are solved, falling with depth despite constant per-step accuracy. Compounding a 100% rate predicts near-total success, not 53%.

A second mechanism better fits `identity`: v2’s train/validation split (`split_problems` in `train_goal_deep.rs`) holds out whole problems stratified by depth and family, but not by wrapper-motif *composition*. Training’s composition layer draws each wrapper motif independently from a

seven-way distribution (Section 5), so a problem with roughly 63 consecutive identity-collision layers, the benchmark’s actual construction, has probability on the order of $0.24^{63} \approx 0$ under training’s own distribution, even though the atomic identity-collision motif itself is common (24% of layers). The benchmark’s monotone-repeat composition is close to unseen during training despite sharing motif vocabulary with it, which is a more specific claim than the deep-OOD legacy-naming issue of Section 7 and applies even to the **deep-ID** split. We read this as the more likely explanation for why a policy with a measured-correct prior at every sampled state still fails to convert that into a solved long-horizon problem: most plausibly, MCTS’s fixed simulation budget cannot accumulate enough confidence margin to commit to the correct branch when the training distribution never rehearsed this specific compositional shape, even though it rehearsed every component of it.

Both measurements use small samples (11 and 2 problems respectively) and should be read as suggestive point estimates, not precise population values. The result is family-dependent and unresolved: compounding error accounts for **power_of_one**’s collapse but not **identity**’s, and we report both rather than only the mechanism that fits.

B Reproducibility Details

The source code, training and evaluation harness, and paper artifacts are available in the [GitHub repository](#). The frozen benchmark is released as a [Hugging Face dataset](#); the paper’s 120-problem evaluation uses its **deep** configuration. The evaluated checkpoint is released as the [LEMMA v3 policy model](#). The archival paper record is available at [doi:10.5281/zenodo.22842820](https://doi.org/10.5281/zenodo.22842820).

B.1 Arms evaluated

Table 10 lists every arm referenced in this paper together with its training configuration, for exact reproduction.

Arm	Guidance	Recovery data	Goal-conditioned
Uniform prior	none	n/a	n/a
Shallow template policy	template, 1-step	n/a	no
Compositional template policy	template, compositional	n/a	no
Trained, v1	learned	yes	yes
Trained, v2	learned	yes	yes
Trained, v3	learned	no	yes
Ablation, with-goal (3 seeds)	learned	no	yes
Ablation, no-goal (3 seeds)	learned	no	no

Table 10. Full arm inventory.

B.2 Network and search hyperparameters

Embedding dimension 256, hidden dimension 512, 8 attention heads, 4 transformer layers, token vocabulary 64, dropout 0.1 during training, **2,272,318 parameters across 69 tensors** (counted directly from a saved checkpoint, not estimated). Optimizer: AdamW, weight decay 0.01. Value loss weight 0.5. Cosine learning-rate schedule, peak 3×10^{-4} , minimum 3×10^{-5} , warmup 5% of total steps. Value-target discount 0.97. Checkpoint selection: every epoch’s weights are saved unconditionally to the primary output path (Section 5), so the file on disk after training is the *last* epoch run, not the epoch with the best validation metric; early stopping triggers once **min_epochs** (12, or

as noted per arm below) has elapsed and **patience** (6) further epochs pass with no improvement in branching-validation accuracy.

MCTS exploration constant $c_{\text{puct}} = 1.41$; evaluation simulation budget 400 per problem (except the budget sweep, Appendix C); maximum search depth is the deepest benchmark problem’s construction depth plus 8. Backup is a simple running mean of backed-up values per node (**visits** += 1, **value_sum** += v), not a discounted or minimax backup. Reaching the goal predicate backs up a fixed terminal value of 1.0; every other leaf (expansion, max-depth cutoff, or no legal moves) backs up the trained value head’s prediction. Final action extraction is deterministic, not temperature-sampled: if any goal-reaching path exists in the tree, the shortest one is taken; otherwise the most-visited child is followed at each step, ties broken toward the lower action index. No Dirichlet or other exploration noise is added to the root priors. Exploration is driven only by PUCT’s own $c_{\text{puct}} \cdot P(s, a) \cdot \sqrt{N(s)} / (1 + N(s, a))$ term.

B.3 Per-arm training data and wall time

Table 11 reports what each run’s own manifest actually recorded; a blank cell means that field was not logged for that specific run rather than being assumed zero or omitted for space. All arms share the network and optimizer settings above; **--recovery-per-problem** and per-depth problem count are the two training-recipe parameters that differ across arms (Table 10).

Arm	Problems validated	Training rows (nominal → distinct)	On-path / recovery	Epochs run	Wall time
v1	2,458	91,110 → 66,754	78,874 / 12,236	13	7,733.5s
v2	1,099	46,107 → 43,462	44,497 / 1,610	n/a	7,812.4s
v3	1,099	44,497 → 42,520	44,497 / 0	7	2,398.9s
Ablation, with/no-goal (each seed)	549	22,247 → 21,408–21,432	22,247 / 0	4	510–642s

Table 11. Per-arm training statistics, taken directly from each run’s own saved manifest. v1’s epoch count is inferred from its best-epoch report (epoch 12, zero-indexed) rather than separately logged; v2’s total epoch count was not captured in its manifest. Ablation wall times vary slightly by seed at fixed epoch count due to normal machine-load variation, not a configuration difference.

Evaluation runtime: the full 120-problem frozen corpus at 400 simulations takes approximately 12 minutes per arm on the parallelized evaluator (roughly seven times faster than the original sequential implementation, Section 7) on a 16-logical-core workstation CPU.

B.4 Seeds

Training data generation seed and evaluation corpus generation seed are independently frozen and distinct. Distinct seeds alone do not guarantee zero overlap: an explicit check compares every generated training input against the held-out corpora by exact string match and discards a collision before it can reach a training example. This check is not vacuous. It caught and removed one exact collision during v2’s data generation, so it is the filter, not the seed choice, that rules out train/test leakage. The ablation study’s per-seed weight-initialization and batch-order seed is an explicit, logged command-line argument (Section 5.1); the train/validation split and the underlying training data are held fixed across seeds within an arm.

C Solve Rate versus Simulation Budget

The main text (Section 8.1) reports every arm at a single simulation budget (400) and is explicit that it does not characterize how solve rate scales with budget. Table 12 closes that gap for one arm, v3, at five budgets from 50 to 800.

Simulations	Total (/120)	<code>identity</code> (/60)	<code>power_of_one</code> (/29)	Meaningful (/89)
50	0	0	0	0
100	45	14	8	22
200	84	36	17	53
400	92	44	17	61
800	96	48	17	65

Table 12. v3’s solve rate against simulation budget, on the same frozen corpus and wrapper-family classification as Table 6. The 400-simulation row (61/89) matches the main text exactly, run independently on the parallelized evaluator (Section 7) as a consistency check.

Two things follow. First, guidance alone is not sufficient at every budget: v3 solves *zero* meaningful problems at 50 simulations despite being the same trained policy that solves 61 of them at 400. A search budget too small to realize a correct policy’s advantage looks, from the outside, identical to having no useful policy at all. Second, returns diminish but do not vanish by 800: `power_of_one` is flat at 17/29 from 200 simulations onward (its remaining 12 problems are not solved by more search at this policy’s fixed accuracy, consistent with Appendix A.1’s compounding-error analysis), while `identity` continues to improve through 800 (48/60, up from 44/60 at 400). We did not extend the sweep past 800 simulations; whether `identity`’s improvement continues, plateaus, or reflects a budget large enough to overcome the compositional-mismatch effect of Appendix A.1 is not established by this experiment.